03-3-build-integrity-notes

accounts-api security assessment · generated from the repository, not hand-edited

> **Supporting write-up for section 3: where the trust root lives, how tag mutation is prevented, what the OIDC scoping denies, and what the signature does and does not prove.**
>
> Assembled by `scripts/build-submission.py` from the repository. Source paths are given above each section.

`docs/02-build-integrity.md`

Section 2 — build and release integrity

Threats: **T1** (CI/CD identity abuse), **T4** (unsigned or mutable artefact). Artefacts: `.github/workflows/build-release.yml`, `iam/github-oidc-trust-policy.json`, `iam/github-oidc-trust-policy-build.json`, `terraform/ecr.tf`.

SBOM: attestation, not build log

`syft` runs against the **pushed image by digest**, so the SBOM describes the artefact that will run, not the workspace that produced it. It is attached with `cosign attest --type spdxjson` to the image digest in ECR.

`upload-artifact` is explicitly disabled. An Actions artefact expires (90 days by default), is not content-addressable, and cannot be retrieved by a responder who holds only a running container's image reference. During a Log4Shell-shaped morning the question is *"which of the 400 running digests contain this package"*, and only a registry-attached attestation answers it:

```
cosign download attestation <repo>@sha256:... | jq -r '.payload|@base64d|fromjson|.predicate' \
  | grep -i log4j
```

Signing: keyless, and what the trust root is

Keyless (Fulcio/Rekor), not a keyed cosign key. One line of justification: a keyed setup moves the problem to *where the private key lives* — and in this pipeline it would live in a GitHub Actions secret, which is readable by any workflow that runs in the repo, i.e. exactly the T1 attacker. Keyless removes the standing secret entirely; the signing key exists for the ~10 minutes of the job and the binding is to the workflow identity, which is the thing I actually want to assert.

Where the trust root lives: the Sigstore public good instance — Fulcio's root CA and the Rekor transparency log, distributed via TUF and pinned in the cluster by the Kyverno policy (`policy/kyverno/policies/verify-image-signature.yaml`). That is a third-party dependency and I am naming it as one. In a real bank rollout I would mirror the TUF root and run a private Fulcio/Rekor (`sigstore/scaffolding`) inside the org so availability and trust are not outsourced; documented in `DECISIONS.md` §6 as cut for time.

Immutable, versioned tags

Three layers, because any one of them alone fails open:

1. **Registry.** `aws_ecr_repository.image_tag_mutability = "IMMUTABLE"` — ECR rejects a second push to an existing tag. This is the load-bearing one.
2. **Pipeline.** Tag is `<VERSION>-<12-char git sha>`, computed from git, never from a PR-controlled input; and a preflight `describe-images` fails the job if the tag already exists, so the failure mode is a red build rather than an opaque `denied` from the registry.
3. **Consumption.** Nothing in the deploy path resolves a tag. `kubectl set image` uses `@sha256:...`, and Kyverno rejects tag-only references at admission (`require-image-digest.yaml`). The tag is a label for humans; the digest is the identity.

`:latest` is never produced. The broken production manifest in section 4 used it, which is how a rollback in that cluster silently became a roll-forward.

What the OIDC scoping denies

The prod role's `sub` is `repo:example-bank/accounts-api:environment:production` — **StringEquals, not StringLike, and no `*` anywhere.** In order of what it kills:

- **Any other repo in the org.** The single most common real-world failure is `repo:example-bank/*`, which means a summer intern's `hackday-bot` repo can assume the production deploy role.
- **Any branch, fork, or tag.** The `sub` for a branch push is `...:ref:refs/heads/x`; for a PR it is `...:pull_request`. Neither matches. So a PR — including one from a fork — cannot reach prod even if it can run a workflow.
- **Any workflow file other than this one.** `job_workflow_ref` is pinned. An attacker who lands a new `.github/workflows/evil.yml` on main still cannot assume the role, because the claim names the file. This is the condition that specifically blunts scenario (b) in `DECISIONS.md` §3.
- **A same-named repo under an attacker-controlled org.** `repository_owner_id` is a numeric, non-reusable id, so re-creating `example-bank/accounts-api` elsewhere after a rename does not work.
- **Long-lived access.** `role-duration-seconds: 900`. A stolen web-identity token is useful for at most 15 minutes, and every use is one `AssumeRoleWithWebIdentity` line in CloudTrail carrying the run id.

Deployment also requires passing the `production` GitHub Environment's reviewers — the environment claim is only minted after that gate — so the AWS condition and the GitHub control reinforce each other rather than duplicating.

The build role is deliberately weaker and separate (different AWS account): it may push to ECR from `main`, and it has no path to the cluster. Compromising the build path gets you an image; the image still has to pass admission.

What the signature proves — and what it does not

This is the part a reviewer will over-read, so plainly:

It proves: that *this* workflow file, on `refs/heads/main` of *this* repo, running under GitHub's OIDC issuer, held the digest at signing time, and that the event is in a public transparency log. If the running digest verifies, it came out of this pipeline and not off someone's laptop or out of a stolen ECR credential.

It does not prove:

- **That the code is safe.** A signature is provenance, not quality. Malicious code that was merged and built is signed just as cleanly as good code. The gates in section 3 are what look at the content; the signature only tells you *whose* content it is.
- **That the source is what you think it is.** The Fulcio certificate binds the workflow ref, not the source tree. Anyone who can merge to `main` — or who can push to `main` directly if branch protection lapses — gets a valid signature. This is precisely why scenario (b) is a serious scenario: signing does not defend against a legitimate committer.
- **That the SBOM is complete.** Syft finds what package managers declare. It misses vendored C, statically linked Go, binaries dropped in during build, and anything installed via `curl | sh` in the Dockerfile. Treat the SBOM as a floor on what is in the image, never a ceiling.
- **That the build was reproducible or hermetic.** The runner has full network access. `mode=max` provenance records *what the build claimed to do*, and buildx records it honestly, but nothing here prevents a build step from fetching an attacker-controlled tarball. A signature over a compromised build is a valid signature over a compromised build.
- **Anything about time-of-use.** Verification at admission checks the artefact identity, not that the artefact is still fit to run. A signed image with a critical CVE published yesterday still verifies. Freshness is the scanner's job, not the signature's.

The one-sentence version for a reviewer: **the signature answers "did our pipeline make this?", not "is this safe to run?"** — and a control that gets read as the latter is worse than no control, because it retires the question.

Local / free-tier equivalent

production (AWS)	what a reviewer can run for free	mapping
ECR in build account `210987654321`	`ghcr.io/<user>/accounts-api` or a local `registry:2`	image push target
ECR `IMMUTABLE` tag setting	GHCR container retention + branch-protected tags	tag mutation prevention (GHCR has no exact equivalent — this is a genuine downgrade, see `DECISIONS.md` §6)
IAM OIDC role for push	`GITHUB_TOKEN` with `packages: write`	short-lived, job-scoped credential
Sigstore public good instance	same	identical; no AWS dependency
`aws eks update-kubeconfig`	`kind create cluster`	deploy target